

Urban Flighter: An Honest Urban-Air Simulator with CFD-lite Hidden Flow and Policy-Visible Sensing

Jaewon Jang

Inha University; VortexyAether

August 2026

Abstract

Urban Flighter is an open-source urban drone simulator that couples real OpenStreetMap (OSM) building geometry, forecast-model inlet wind, and a browser cockpit with a headless research environment. The design goal is not a new Navier–Stokes solver. It is an honest contract: hidden urban-flow approximations may drive dynamics, energy, and metrics, while a policy or baseline may observe only mission context, known inlet wind, onboard relative-air estimates, and deterministic geometry LiDAR. We document the three public flight views (2D, 3D Lite, True 3D Wind overlay), the potential-flow CFD-lite field with wall damping and empirical wake correction, and UrbanFlow Gym, a NumPy environment that is RL-ready but not trained. A fully offline showcase case on a synthetic fixture shows the expected qualitative result: unconstrained goal-seeking collides, geometry-safe A* succeeds, and an inlet-aware heuristic is not automatically energy-optimal.

1 Introduction

Urban air mobility and small-UAV routing need environments where buildings, wind, sensing, and energy interact. Many simulators either hide those interactions behind game physics or leak privileged flow fields into the learning agent. Urban Flighter takes a third position: keep the cockpit watchable, keep the geometry real when a city is loaded, and keep the research contract explicit.

The software is already a working full-stack prototype:

- a FastAPI backend that fetches OSM footprints and Open-Meteo current conditions, then computes CFD-lite fields;
- a React/Three.js frontend with 2D and 3D flight, LiDAR-inspired returns, and a Gym episode inspector;
- a headless UrbanFlow Gym core with leakage guards and deterministic baselines.

This report is a system paper / technical report, not a claim of a new turbulence model or a trained policy. All quantitative numbers in Section 7 are produced by scripts/run_oss_showcase.py and stored under docs/showcase/.

2 Related systems and positioning

AirSim, FlightGoggles, and related aerial simulators emphasize photorealism or vehicle dynamics. Standalone CFD packages emphasize field accuracy but are not interactive flight products. Urban Flighter sits between them: OSM-scale urban geometry, cheap hidden flow, and an RL-shaped observation contract.

Closest internal relatives are the project’s own AeroJAX 2D incompressible Navier–Stokes snapshots (optional offline engine) and the browser CFD-lite B grid used for flyable dynamics. The public cockpit does not treat bundled True 3D $u/v/w$ streamlines as the flyable field.

3 System architecture

Backend. `backend/` serves geometry, weather, and flow. `/flow-fields/2d` is the primary live endpoint: a streamfunction / potential-flow-ish grid with wall damping and an empirical wake correction (“CFD-lite B”). Successful non-empty responses are registered as an immutable content-addressed snapshot `urbanflow.live_scenario.v1`.

Frontend. Three public modes:

- 2D: north-up canvas, 120 Hz fixed-step motion, swept-circle clearance, 180-ray scan.
- 3D Lite: metre-scale quadcopter, same horizontal live grid, building/ground LiDAR, presentation-only city dressing.
- True 3D Wind: the same flyable city plus a bundled 3D potential-flow $u/v/w$ overlay (Gangnam demo). Overlay $\neq$ flight dynamics.

Energy proxy. Unless a more detailed motor model is requested, step energy uses relative airspeed

$$e_t = \|\mathbf{v}_{\text{ground},t} - \mathbf{w}_{\text{local}}(\mathbf{x}_t, t)\|^2 \Delta t. \quad (1)$$

The local wind $\mathbf{w}_{\text{local}}$ is a hidden sample. The actor may receive only an onboard estimate and the known inlet.

Honesty labels. The product UI and this paper use the same vocabulary: CFD-lite $\neq$ Navier–Stokes; SIM odometry / no loop closure; UrbanFlow Gym / NOT TRAINED; full flow access = NO.

4 CFD-lite hidden dynamics

4.1 Potential-flow slice

A 2D slice solves a discrete Laplace problem for a streamfunction or potential on a masked grid, then recovers $\mathbf{u} = \nabla\phi$ (or the streamfunction equivalent). Inlet / far-field velocity is prescribed; building cells are solid. This produces cheap acceleration around corners (Fig. 1).

It cannot represent viscous separation, vortex shedding, or turbulence. Empirical wall damping and downstream wake patches are therefore added for drone-risk realism and must be labelled as such.

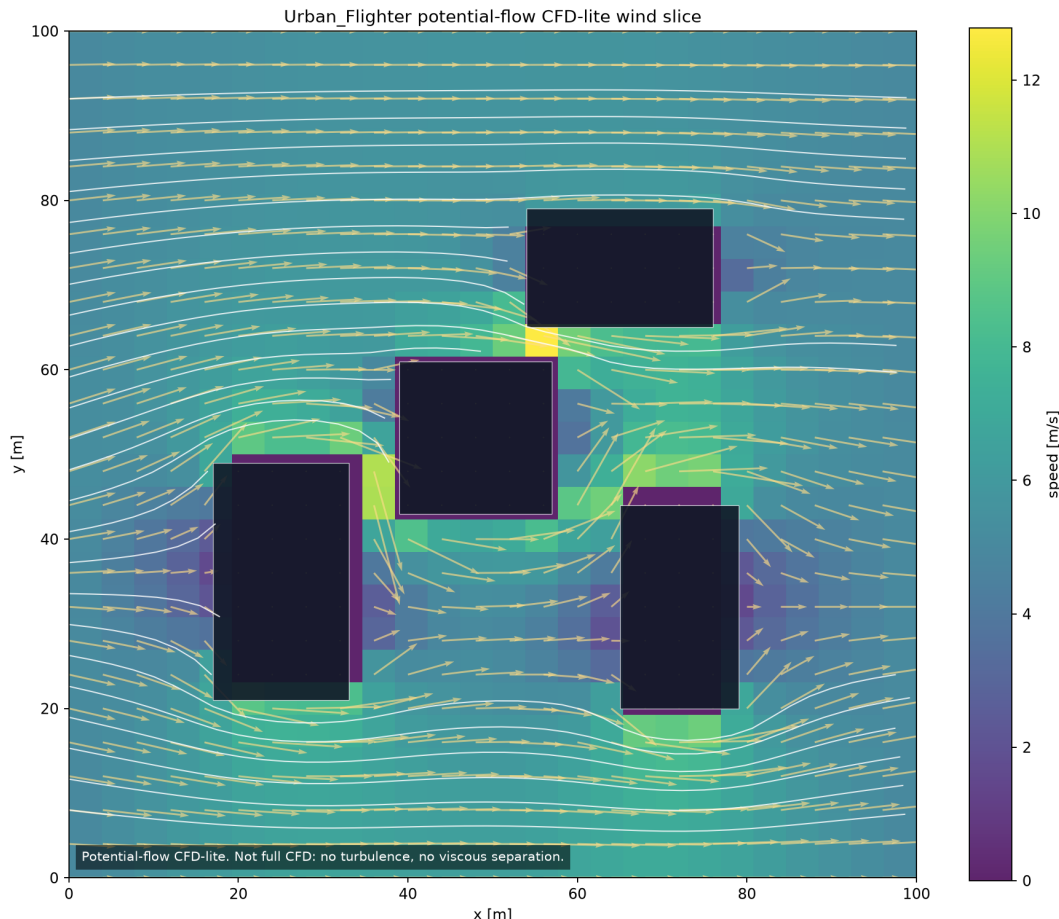


Figure 1: Offline toy-city potential-flow CFD-lite showcase. Five prisms, 5 m/s westerly inlet, 26×26 grid, 348 iterations, residual 9.84×10^{-5} . Mean speed 4.96 m/s, max speed 12.8 m/s. Not Navier–Stokes.

4.2 True 3D overlay

A voxel potential-flow field supplies u, v, w streamlines for visualization. The flyable 3D aircraft still samples the live horizontal CFD-lite grid. No Navier–Stokes residual is claimed for either field.

5 Sensing and the actor contract

Frontend LiDAR prototypes use a frozen five-value-per-ray layout

$$(d_x, d_y, d_z, \hat{r}, h),$$

local direction, normalized range, and hit flag. 3D default: 600 Fibonacci-sphere samples, 120 m max range. 2D default: 180 rays, 180 m.

UrbanFlow Gym uses a separate 33-scalar actor observation: own motion, goal context, fixed-angle geometry LiDAR, known inlet, previous action, and an onboard relative-air estimate. Automated leakage guards reject full grids, exact wakes, future samples, and provider objects in the actor channel. A privileged critic helper exists and is never returned by reset/step.

Table 1: UrbanFlow Gym synthetic-fixture evaluation. Policy status: NOT TRAINED. Privileged flow access: false. Hidden dynamics: synthetic potential-flow-ish wake proxy, not Navier–Stokes.

Baseline	Success	Mean path (m)	Rel-air energy	Collision eps.
Direct goal	0/3	23.5	180	3/3
Shortest-path A*	3/3	142.1	1236	0/3
Inlet-aware A*	3/3	141.9	1303	0/3

6 UrbanFlow Gym

The core environment is NumPy-only. Gymnasium / Stable-Baselines3 / Torch are optional extras and are not imported on the default path. Action is a bounded 2D local-guidance command in the vehicle frame, not motor RPM.

Three deterministic baselines share the public mission context only:

1. Direct goal: steer toward the goal (no obstacle planner).
2. Shortest path: geometry-safe A* + waypoint tracking.
3. Inlet-aware: A* with known-inlet feed-forward. Still not privileged-grid access.

Live evaluation binds to the exact UI-registered OSM scenario. The showcase below uses the synthetic rectangular fixture so that open-source CI needs no network.

7 Showcase case

Command (repo root, no network):

PYTHONPATH=backend python scripts/run_oss_showcase.py

Artifacts: docs/showcase/.

7.1 Potential-flow field

Toy city, five buildings, inlet from 270° at 5 m/s (Fig. 1).

7.2 Baseline comparison

Three held-out fixture seeds {10007, 10009, 10037}, horizon 240 steps, $\Delta t = 0.25$ s. Table 1 and Figs. 2–3.

Reading. The showcase is a contract test, not a claim that wind-aware planning wins. A successful OSS demo must show (i) collisions when geometry is ignored, (ii) success when it is not, (iii) no privileged-grid leakage, and (iv) honest labels.

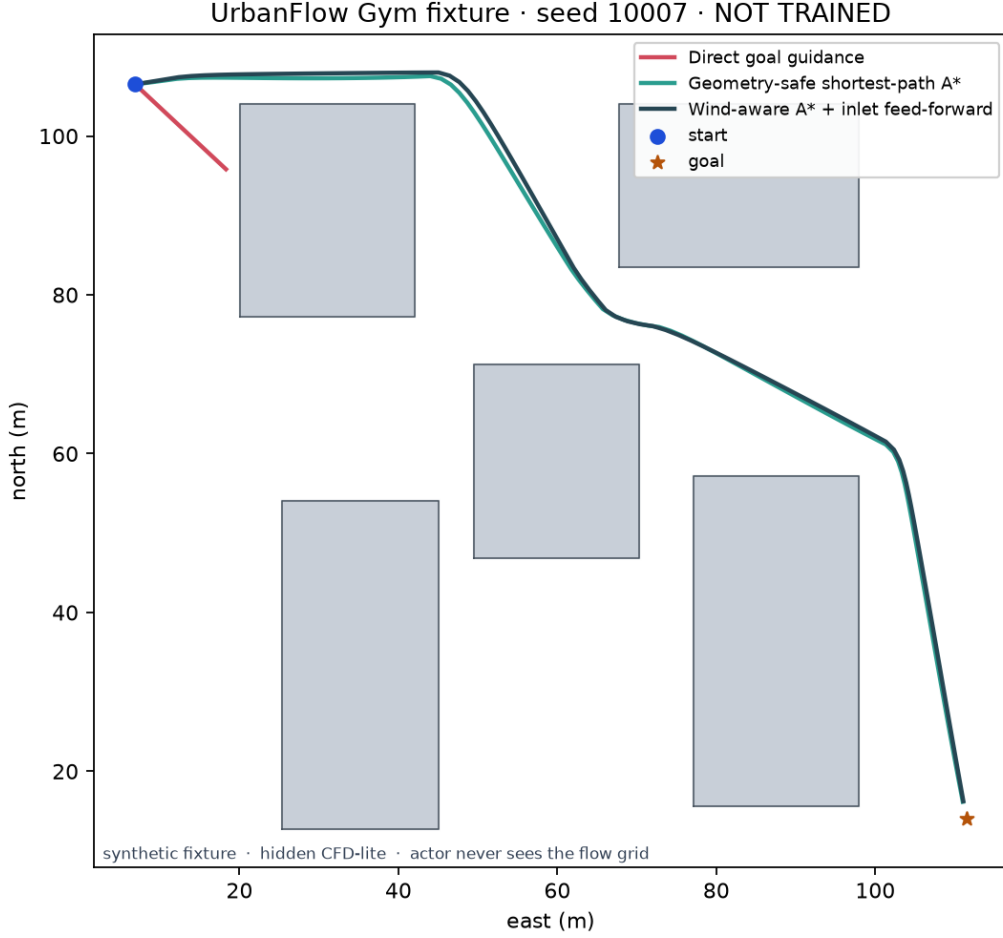


Figure 2: Seed 10007 rollouts on the synthetic fixture. Direct goal collides. Both planners reach the goal without building hits. Actor observation dimension is 33.

8 Limitations

- CFD-lite and voxel potential flow are not LES/RANS/DNS.
- True 3D Wind is a visualization overlay.
- LiDAR and rolling maps are simulator rays plus odometry, not hardware SLAM.
- No PPO/SAC training is shipped; “RL-ready” $\neq$ “trained.”
- Live OSM/weather paths need network; the committed showcase does not.
- Presentation trees/roads never enter physics, sensors, Gym, or scenario hashes.

9 Reproducibility

- Code: https://github.com/VortexyAether/Urban_Flighter
- License: MIT.

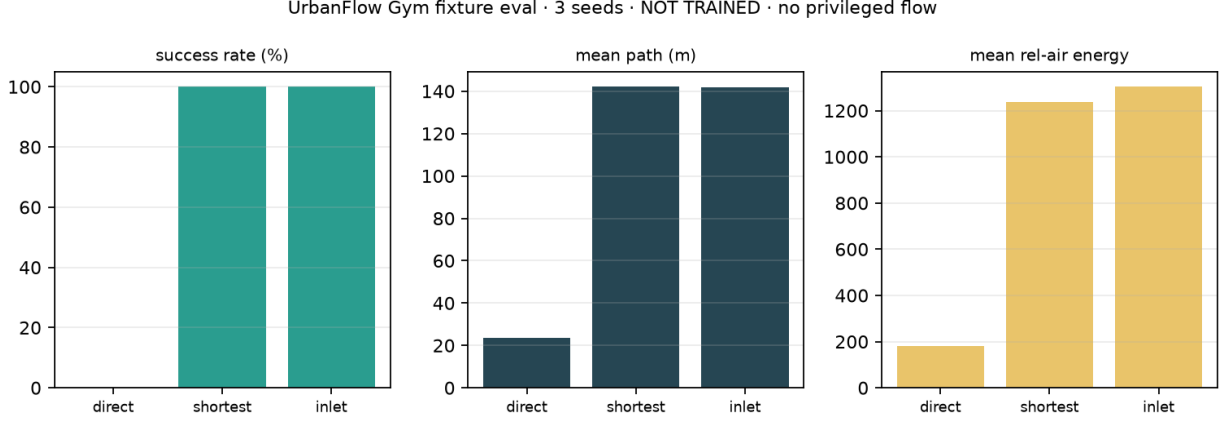


Figure 3: Aggregate fixture metrics. Inlet-aware is not energy-better than shortest path on this set; we report the regression rather than force a narrative.

- Showcase seed and command: Section 7.
- Evaluation id of the reported JSON: 24ec026f3352e4f9.

10 Conclusion

Urban Fighter is released as an open urban-air research cockpit: real cities when requested, cheap hidden flow, and a deliberately narrow actor contract. The first public case study is intentionally boring and checkable. That is the point.

A Software map

backend/ FastAPI and CFD-lite services; backend/urbanflow_gym/ headless env; frontend/ React 19 + Three.js; scripts/run_oss_showcase.py this paper’s case; CLAUDE.md contributor contract.